

Paso 10 - Clases con métodos `static` | Ejercicios

Desde aquí, cada ejercicio son **dos archivos**: la clase con los métodos, y el programa con el `main` que la usa.

```
javac Clase.java Programa.java
java Programa
```

Y la regla de siempre: **el archivo se llama igual que la clase**.

► **Plantillas:** [plantillas/](#) — un `.java` por ejercicio, con el enunciado, la salida esperada y los `// TODO` donde va tu código. Si las usas, el archivo ya viene creado (`B10_1_MasNotas.java`) y **ese** nombre manda sobre el que diga el enunciado.

✓ **Solucionario:** [soluciones/](#) — el código de cada ejercicio, la salida real y por qué se resuelve así. Ábrelo **después** de intentarlo.

Cómo se lee cada ejercicio: el título dice qué hay que lograr; el bloque de abajo es la **salida esperada**, o sea lo que tiene que aparecer en tu pantalla (en los programas que preguntan cosas, lo que va después de la pregunta es lo que **escribe el usuario**); las citas como esta son pistas. Si no sabes por dónde partir, abre la plantilla: los `// TODO` numerados son los pasos, en orden. Compila y ejecuta después de cada uno.

B10.1 - Agrandar `Notas`

Qué practicas: agregar métodos nuevos a una clase existente, y llamar a un método de la misma clase sin prefijo.

Agrega a tu clase `Notas`:

```
public static double minima(double[] notas)
public static int contarAzules(double[] notas)    // notas sobre 5.0
public static double porcentajeAprobacion(double[] notas)
```

Pruébalas desde tu programa del paso 10 (o desde la plantilla `B10_1_MasNotas.java`) con el arreglo `{5.5, 6.2, 3.8, 4.5}`:

```
Minima: 3.8
Azules: 2
Aprobacion: 75.0%
```

`porcentajeAprobacion` puede **usar** `contarRojas`, que ya existe. Un método de la clase llamando a otro de la misma clase: sin prefijo.

B10.2 - La clase `Conversor`

Qué practicas: escribir una clase de utilidades sin `main`, con constantes y métodos `static`, igual que `Math`.

Crea `Conversor.java`, sin `main`, con:

```
public static final int UF = 39000;           // valor aproximado, en pesos

public static double celsiusAFahrenheit(double c)    // c * 9 / 5 + 32
public static double kmAMillas(double km)            // km * 0.621371
public static double pesosAUf(int pesos)
public static int ufAPesos(double uf)
```

Y un segundo archivo con el `main` que las pruebe:

```
25 C son 77.0 F
5 km son 3.1 millas
1500000 pesos son 38.46 UF
2.5 UF son 97500 pesos
```

Cuidado con escribir `c * (9 / 5)`: los paréntesis hacen que Java divida primero **dos enteros**, `9 / 5` da 1, y 25 °C te quedan en 57.0 °F en vez de 77.0. Es la trampa del paso 3, escondida en una fórmula real.

B10.3 - La clase `Numeros`

Qué practicas: trasladar métodos que ya escribiste a su propia clase, y llamarlos desde el programa que los usa.

Crea `Numeros.java` con los métodos que ya escribiste sueltos en el paso 9:

```
public static boolean esPar(int n)
public static boolean esPrimo(int n)
public static long factorial(int n)
public static int sumaDigitos(int n)           // el del ejercicio B3.3
```

Desde el `main`, imprime una tabla del 1 al 20 con todo junto:

```
n  par    primo  factorial  suma digitos
1  no     no     1           1
2  si     si     2           2
...
```

B10.4 - Desafío: la clase `Texto`

Qué prácticas: recorrer un `String` carácter por carácter con `charAt` y `length()` para construir o analizar texto.

Crea `Texto.java` con:

```
public static String alReves(String texto)
public static boolean esPalindromo(String texto)
public static int contarVocales(String texto)
public static String inicialesDe(String nombreCompleto) // "Ana Perez Soto" ->
"APS"
```

```
alReves("DUOC") -> COUD
esPalindromo("reconocer") -> true
contarVocales("murcielago") -> 5
inicialesDe("Ana Perez Soto") -> APS
```

Pistas: `texto.length()`, `texto.charAt(i)`, `texto.toLowerCase()`, `texto.charAt(i) == 'a'`. Para las iniciales: la primera letra, y toda letra que venga justo después de un espacio.

B10.5 - Preguntas para responder por escrito

Qué prácticas: poner en palabras propias por qué una clase no necesita `main`, la diferencia entre `public` y sin modificador, y qué distingue a `Math` de `Scanner`.

Este es el único ejercicio **sin plantilla**: se responde en un documento, o en comentarios al final de cualquiera de los archivos de este paso. Escribe tu respuesta **antes** de mirar la del solucionario.

1. ¿Por qué `Notas` no tiene `main`? ¿Qué pasa si intentas `java Notas`?
2. ¿Por qué los métodos de `Notas` llevan `public` y los del paso 9 no lo necesitaban?
3. `Math.sqrt(16)` y `Notas.promedio(notas)` se escriben igual. ¿En qué se parecen `Math` y `Notas`?

4. **Scanner** **sí** necesita **new** y **Math** **no**. ¿Cuál es la diferencia entre las dos? (Nadie espera que lo sepas del todo: es la demo 1. Escribe lo que sospechas y guárdalo para compararlo después.)
-

Cuando termines, sigue con el [paso 11](#).